CITY UNIVERSITY OF HONG KONG (DONGGUAN)

Course code & title	:	CS5351 Software Engineering
Session	:	Semester A 2025/26
Time allowed	:	Two hours

This paper has 13 pages (including this cover page).

1. This paper consists of 3 parts.
2. Answer ALL questions.
3. Write your answers **in the answer box**.
4. This question paper should **NOT** be taken away.
5. Write your answers in English

*This is a **closed-book** examination.*

No materials or aids are allowed during the whole examination. If any unauthorized materials or aids are found on a student during the examination, the student will be subject to disciplinary action.

	Part I	**Part II**	**Part III**			Total
Max	10	30	20	20	20	100
Mark						

Student Name: _____

Student ID: _____

Seat Number: _____

Part I. True/False Questions (10 points total, 1 point each)
Instructions: Read each statement below. Mark 'T' if the statement is True or 'F' if the statement is False in the space provided.

1. In the Scrum process, the main purpose of the Daily Scrum is for development team members to individually report their work progress to the project manager.
[ ]

2. The requirement "The project must go live within 4 months" belongs to the non-functional requirement category of constraints.
[ ]

3. In software architecture, one of the main advantages of a Microservices architecture compared to a Monolithic architecture is higher availability, because the failure of a single service does not bring down the entire system.
[ ]

4. Technical debt refers to the "quick and dirty" shortcuts developers intentionally take to meet short-term deadlines, and it only includes debt that is incurred intentionally.
[ ]

5. Modern code review is an informal, tool-based, asynchronous review practice that focuses on reviewing code changes.
[ ]

6. In Scrum, Project Velocity is a measure of the amount of work a team can complete in a single Sprint, and it is often used to forecast how many Sprints are needed to complete the remaining work.
[ ]

7. The four main steps of requirements engineering are: Elicitation, Specification, Validation, and Coding.
[ ]

8. According to the "Managing Technical Debts in Practice" survey, using comments in the code is considered an effective way to track technical debt.
[ ]

9. One of the best practices for modern code review is to combine multiple, unrelated changes (like bug fixes, refactoring, and new features) into a single pull request to be reviewed all at once.
[ ]

10. In software architecture, a Monolithic architecture is relatively simple to develop initially and often has high performance due to in-memory communication between components.
[ ]

Part II: Multiple-Choice Questions (Total: 30 points)
Instructions: Read each question and its options carefully.

1. Select ALL options that correctly answer the question.
2. Each question has at least two correct options.
3. Grading: Each question is worth 3 points. You will receive 1 point for each correct option selected. To receive the full 3 points, you must select all correct options and no incorrect options. No points will be given if any incorrect option is selected.

Question 1: A software team manages its work as follows:

1. The team maintains a "Project Backlog" containing all requirements, sorted by priority.
2. After completing their current task, developers pull the highest-priority task from the "Project Backlog" to work on.
3. At the end of each month, all tasks marked as "Done" are bundled into a new release.

According to the core principles of the scrum framework, which of the following key practices or artifacts are missing from this team's process?

[ ]

(A) Sprints
(B) Product Backlog
(C) Sprint Backlog
(D) Measuring Project Velocity
(E) Continuous Integration

Question 2: Which of the following statements about the characteristics and practices of modern code review are correct?

[ ]

(A) MCR is a lightweight, tool-based (e.g., GitHub, Gerrit) process that is typically conducted asynchronously.
(B) The core of MCR is to review code *changes* (i.e., a patch or delta), rather than reviewing an entire module or system at once.
(C) MCR is a formal, synchronous meeting where reviewers play predefined roles (e.g., moderator, recorder).
(D) MCR best practices include keeping change sizes small and having a small number of reviewers (e.g., 1 to 5).
(E) The *sole* motivation for MCR is to find defects before code enters production.

Question 3: During the requirements engineering phase for a Mobile Test System (MTS), stakeholders identified two key quality attributes:

1. Availability: "Failure of a single invigilator's device should not affect the execution of the test."
2. Performance: "The architecture of MTS should be scalable enough to cater for many test sessions to be held with overlapping periods and in areas overlapping with one another."

Which of the following judgments about these requirements are reasonable in the context of RE?

[ ]

(A) These two quality attributes (availability and performance) may conflict, representing a trade-off that the architect must balance.
(B) The term "scalable enough" in requirement 2 is a common error in requirement definition, specifically ambiguity.
(C) A focus group or a facilitated workshop would be an effective elicitation technique to discuss these requirements with diverse stakeholders (e.g., invigilators, students, administrators).
(D) These two requirements are functional requirements because they describe what the system must do.
(E) The best tactic to address the availability requirement is "Hide Information".

Question 4: An architect is designing a highly reliable system. A key architectural driver is high availability, as stakeholders are concerned about the system becoming unresponsive during an operation. To address high availability, an architect can employ various architectural *tactics*.
Which of the following are effective tactics for availability?

[ ]

(A) Heartbeat and Ping/Echo
(B) Active Redundancy and Spare
(C) Hide Information and Semantic Coherence
(D) Authenticate Actors and Limit Access
(E) Introduce Concurrency and Scheduling Policy

Question 5: During a project retrospective, a team identified several issues that increased maintenance costs. Please classify the following descriptions based on the dimensions of technical debt (e.g., Intentionality, Time Horizon).

1. Issue 1: To meet an urgent release deadline, the team deliberately skipped all unit tests for one module, planning to add them "later."
2. Issue 2: A junior programmer, due to inexperience, wrote complex code that did not follow coding standards, causing others to work more slowly in that module.
3. Issue 3: The team made a strategic decision for the system to only support iOS in its first version, with a plan to refactor for Android support in 3 years if the product is successful.

Which of the following classifications are correct?

[ ]

(A) Issue 1 is Unintentional technical debt.
(B) Issue 2 is Unintentional technical debt.
(C) Issue 2 is Intentional technical debt.
(D) Issue 1 is Intentional and Short-Term technical debt.
(E) Issue 3 is Intentional and Long-Term technical debt.

Question 6: In high-quality code design, several fundamental principles are essential. Regarding abstraction, information hiding, and encapsulation, which of the following statements are correct?

[ ]

(A) Abstraction is the separation of interface from implementation.
(B) Information Hiding is the practice of minimizing the implementation details exposed in an interface.
(C) Encapsulation is the restriction of client code from accessing internal implementation details.
(D) The primary purpose of Abstraction is to ensure the code runs as fast as possible.
(E) Encapsulation means all class members (data members) must be public.

Question 7: The SOLID principles are core guidelines for object-oriented design. Regarding the open-closed principle and the liskov substitution principle, which of the following statements are correct?

[ ]

(A) OCP states that code should be "open for extension, but closed for modification".
(B) A typical way to follow OCP is by using if-else or switch statements to add new functionality.
(C) LSP means that a subclass must be able to replace its superclass without breaking the program's logic.
(D) If a subclass overrides a parent's method but implements it as an empty (dummy) method, this likely violates LSP.
(E) LSP means that a superclass should always be able to replace its subclass.

Question 8: Fuzzing is an automated software testing technique. Regarding fuzzing and its tools (like Android Monkey or AFL), which of the following statements are correct?

[ ]

(A) The main purpose of Fuzzing is to find program vulnerabilities (like crashes) by providing random or mutated inputs.
(B) Android Monkey is a fuzzing tool that simulates random user events (like clicks and touches) on an Android application.
(C) AFL (American Fuzzy Lop) is a feedback-based and mutation-based fuzzer.
(D) Fuzzing can only be used to test programs written in C/C++.
(E) All test cases generated by fuzzing guarantee 100% code coverage.

Question 9: When a program crashes on a very large or complex input (e.g., a long HTML file or a long event sequence), delta debugging is a useful technique. Which of the following statements about delta debugging are correct?

[ ]

(A) The goal of delta debugging is to automatically find and fix the bug in the code.
(B) The core idea of delta debugging is to minimize the failure-inducing input.

(C) The technique works by systematically and iteratively removing parts of the input and checking if the program still fails (crashes).
(D) Delta debugging can only be used to minimize text file inputs.
(E) A smaller, reproducible input helps developers locate the bug more easily.

Question 10: A test oracle is a mechanism used to determine whether a program's execution result is correct. Regarding different types of test oracles, which of the following statements are correct?

[ ]

(A) A program crash or deadlock can be considered a generic test oracle.
(B) A Regression test oracle checks for new errors by comparing the output of a new program version against the output of a previous version.
(C) Metamorphic testing checks for errors by verifying if a specific relationship (Metamorphic Relation) holds true between multiple test inputs and their outputs.
(D) All test oracles must be fully automated.
(E) Using *assertEqual()* in JUnit is an example of a generic test oracle.

Part III: Open-Ended Question

Question 1: Comprehensive Analysis of Software Quality Assurance, Testing, and Debugging (20 points)

Case Study

A company is developing a new "Coupon" module for its existing e-commerce platform. The team uses an Agile development and CI/CD (Continuous Integration / Continuous Deployment) process.

Although their CI/CD pipeline includes automated Unit Tests, numerous problems still appear in the production environment after new features (e.g., "Spend $100, get $10 off" or "20% off specific items") go live. The specific symptoms include:

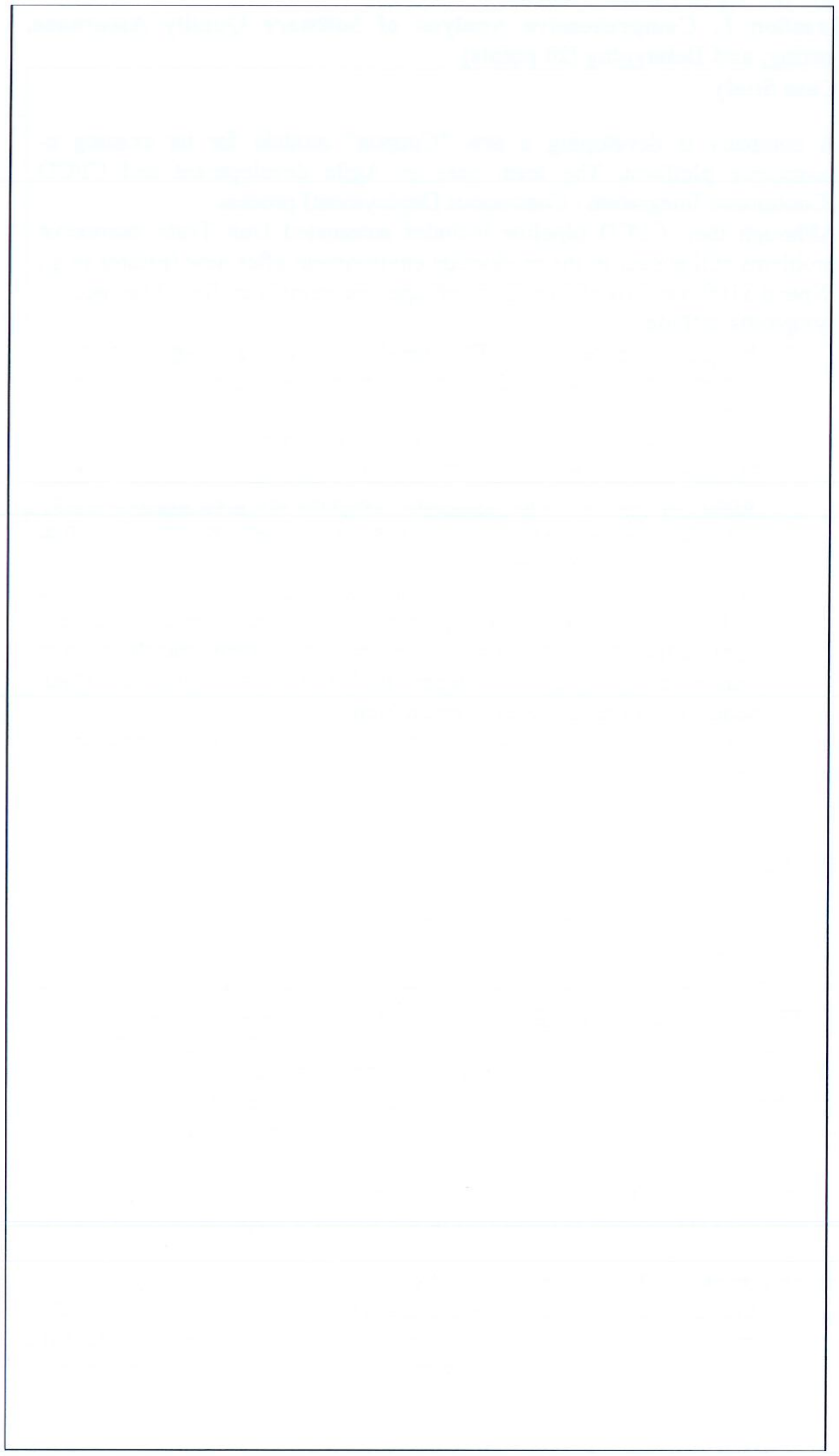
1. Frequent Regressions: The newly developed coupon features sometimes cause old (and supposedly unrelated) coupon functionalities to fail.
2. Brittle Code: Developers report that the coupon module's code is "fragile and breaks easily." To fix one bug (e.g., a calculation error when "Spend/Get" and "Discount" coupons are used together), they have to modify several files. These modifications, in turn, often introduce new, unexpected bugs.
3. Hard to Debug: Crash reports from the production environment are difficult to reproduce. One report states: "I added 3 items to the cart, applied the '20% off' coupon, then removed 1 item, and the system crashed at checkout." Developers find it difficult to pinpoint which sequence of actions caused the problem.

As a graduate of an SDE course, you are hired as a technical consultant to analyze and resolve these issues.

(a) **(6 points)** Code Quality Diagnosis
Based on the symptoms described (brittle code, frequent regressions, difficult to modify), diagnose the potential deep-seated problems at the **code level** within the "Coupon" module.
Select at least two principles from the SOLID principles or core design principles (e.g., Coupling, Cohesion) that this module most likely violates. Explain why these violations would lead to the observed symptoms.

(b) **(8 points)** Improving the Automated Testing Strategy
The team's existing unit tests are clearly insufficient for catching these complex bugs. Propose a more robust automated testing strategy.
Identify two different automated testing techniques that can be integrated into the CI/CD pipeline. Explain how each technique would help the team discover the problems described in the scenario (especially "regressions" and "crashes").

(c) **(6 points)** Production Debugging Process
For the complex, hard-to-reproduce crash reports that have already leaked into production (like the "I added 3 items... then it crashed" report), describe a systematic debugging process that is more efficient than "random guessing" to help developers isolate the problem.



Question 2. Code with Quality (20 points)

Case Study – "CityBus+ Scheduling Module"

CityBus+ is developing a new bus scheduling module. The goal is to compute the estimated travel time between stops and log the daily operation details. A prototype developed by a junior team member is shown below. You are required to review this module against fundamental design principles.

```
class Scheduler:
    def __init__(self):
        self.speed = 40
        self.buffer = 5
        self.logFile = "/var/log/buslog.txt"
        self.records = []

    def schedule(self, route, distance, isPeak):
        if isPeak:
            speed = self.speed - 10
        else:
            speed = self.speed

        time = distance / speed
        time = time + self.buffer

        self.records.append((route, time))

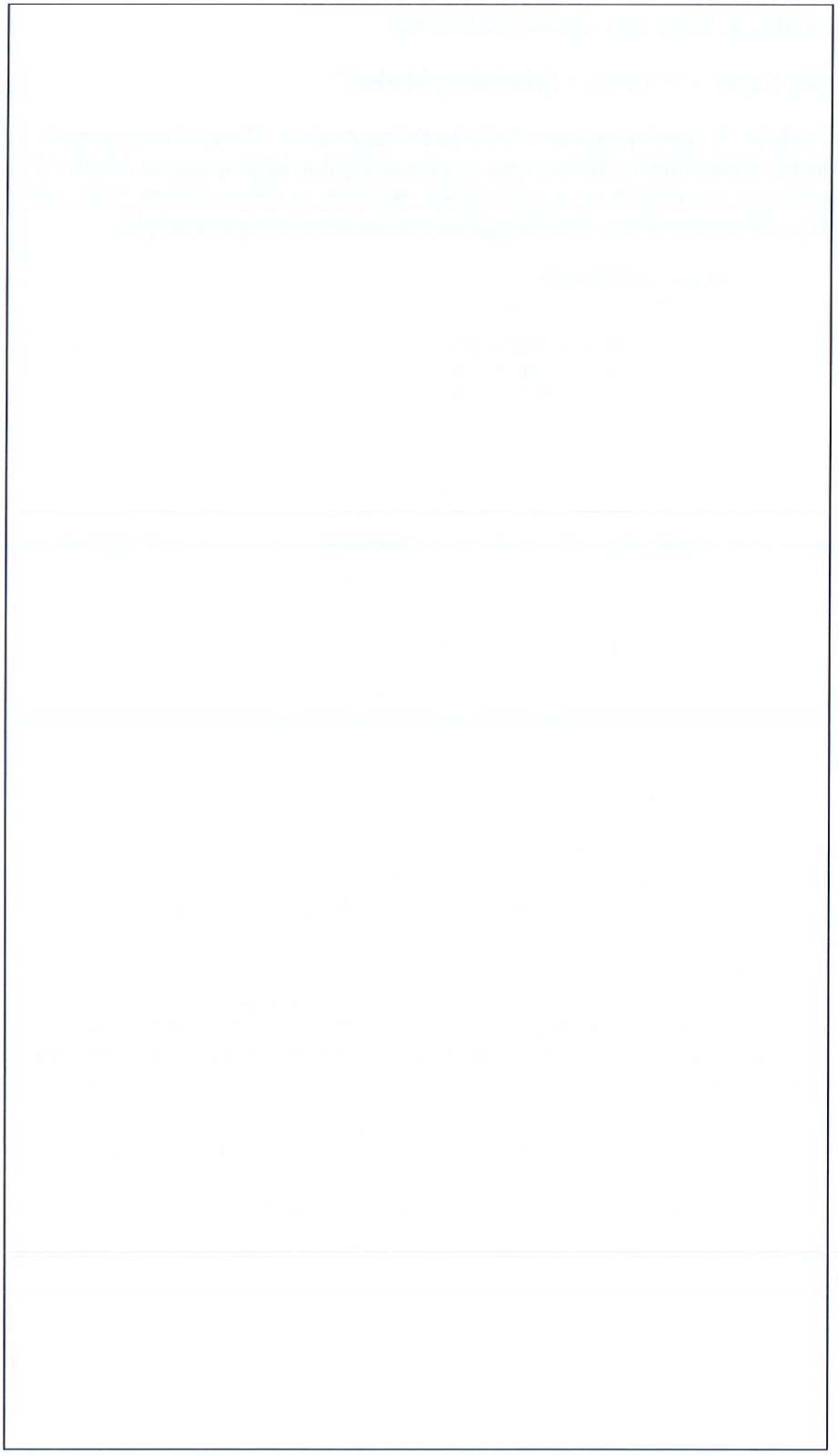
        f = open(self.logFile, "a")
        f.write(f"{route},{time}\n")
        f.close()

        return time
```

(a) (8 points) Identify four major structural or semantic code quality issues (Code Smells) in the snippet. For each issue, clearly explain how it directly compromises the modularity, maintainability, or testability of the application.

(b) (6 points) Based on the code's current structure, identify two distinct design principles concerning modularity and managing change (e.g., related to responsibility, information flow, or dependency) that are violated. For each, specify the exact lines of code that demonstrate the violation and articulate the resulting engineering challenge.

(c) (6 points) Propose two concrete refactoring actions that utilize patterns like Extract Method or Extract Class to fundamentally improve the design. For each action, describe the precise mechanical change and name the specific design principle or quality issue it successfully addresses.



Question 3. Software Testing (20 points)

Case Study – "EasyBook Reservation System"

The EasyBook system manages seating availability using the core reserve API. The business rules governing a successful reservation are strict and detailed.

API: reserve(seatID, userType, hasPaid)

Rules:

1. seatID must be in 1–300.
2. userType $\in$ {Regular, Member}.
3. Members may reserve without payment.
4. Regular users must pay first.
5. If a seat is already reserved $\rightarrow$ reject new reservation (State-based failure).

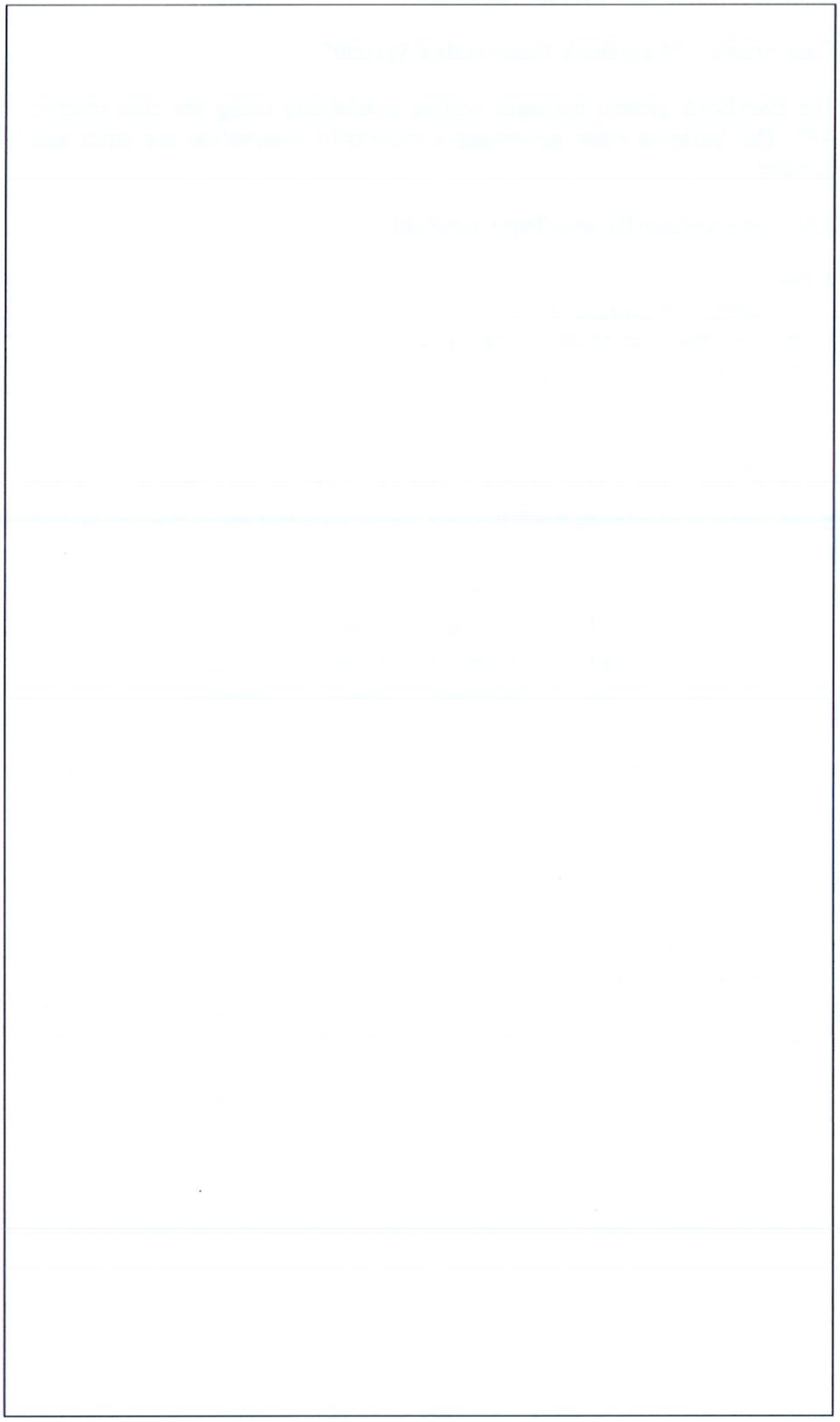
A junior tester developed the following preliminary test suite. You are tasked with evaluating the quality and coverage of this suite.

Test ID	seatID	userType	hasPaid	Expected
T1	10	Regular	True	success
T2	400	Member	False	success
T3	20	Regular	False	success

(a) (6 points) Critically evaluate the test cases (T1, T2, T3) and identify three distinct flaws (e.g., input defect, oracle defect, missing coverage). For each flaw, explain why it fails to effectively validate the reservation system's requirements.

(b) (6 points) Using weak normal equivalence class testing, derive a set of input conditions. Identify two valid and two invalid equivalence classes that cover all input parameters based on Rules 1, 2, 3, and 4. Provide one distinct test case for each of the four classes.

(c) (8 points) Given the existing tests result in Statement coverage = 92% but Branch coverage = 48%, analyze the gaps. Identify two branches derived from the Rules (1-5) that are highly likely to be uncovered. Propose one minimal test sequence (or single test case) for each missing branch, justifying how your proposed test guarantees coverage of that specific logic path.



- END -

Appendix:

Answer for Question _____